

RAG Information Retrieval System — Assessment Report

Apple Inc. Form 10-Q, fiscal quarter ended June 25, 2022

Prepared for Smart Data Solutions · Software Engineer I, Machine Learning

Repository: https://github.com/Gopinath14141/SmartData_Solutions_RAG_Assessment

1. Executive summary

This report describes a retrieval-augmented generation system that answers questions about the text, tables, and figures of a single SEC filing, and does so without fabricating anything.

The interesting problem in this assignment is not the retrieval architecture. It is that the source document's tables do not survive extraction intact, and a system built on the raw extraction will confidently report numbers under the wrong period. Most of the engineering effort went there.

Measured results, over 33 questions with ground truth read from the filing, reproduced across two consecutive runs:

Retrieval hit rate@8	96.4%
Recall@8	91.1%
MRR	0.724
Answer accuracy (23 checkable questions)	91.3%
Refusal accuracy	97.0%
Hallucination rate	0.0%
Citation accuracy	82.1%
Median latency	2.5 s

The system answered every one of the five unsupported questions correctly by declining.

Two failures remain, both diagnosed rather than hidden, and two attempts to improve results by tuning were measured and **made things worse** — recorded here because the negative results are as informative as the positive ones.

2. Problem understanding

The assignment asks for a RAG system answering questions about text, tables, and figures in a supplied PDF, submitted as a repository with a separate report explaining approach, design decisions, assumptions, challenges, and improvements.

Two things about the supplied document shaped everything that followed, and neither was apparent before inspecting it.

Its tables are broken in a specific, recoverable way. The file is a born-digital PDF produced by EDGAR's HTML-to-PDF converter. Every table cell — including currency symbols — is emitted as an independently positioned text run, and the detected table region excludes the period headers above it.

It contains no figures. Not "few". One raster image across 28 pages: the Apple logo.

The second fact creates a genuine tension with the assignment, which requires figure question answering. Section 9 explains how it was resolved.

3. Requirements

Requirement	How it is met
Answer questions about text	Section-aware chunking of narrative; 100% hit rate on 10 text questions
Answer questions about tables	Header recovery, column collapse, units attachment; 93.3% hit rate, 86.7% accuracy on 15 table questions
Answer questions about figures	Image extraction with provenance, multimodal page reading, and a document-level figure inventory; 100% hit rate on 3 figure questions
Well-structured, maintainable code	11 packages with single responsibilities, typed throughout, 113 tests, lint-clean
Explain approach and reasoning	This report, plus three design documents in docs/

4. PDF analysis

Measured with PyMuPDF before any architecture was proposed. Full detail in [docs/pdf-analysis.md](#) .

Property	Value
Pages	28
Producer	EDGAR Filing HTML Converter
Extractable text	70,031 characters
Pages with no text layer	0 — OCR is not needed anywhere
Tables detected	31
Embedded raster images	1 (46×56 JPEG — the Apple logo)
Vector drawing operations	5,215 — all table rules, not charts
PDF outline entries	0

The four findings that drove the design

1. Page numbers are not what they appear. The footer establishes that PDF page 4 is printed page 1 — a constant offset of three, verified at three independent points. The table of contents cites printed numbers. Conflating them puts every citation the system emits three pages out.

2. Table headers fall outside the table. On page 4 the detected bounding box starts at `y=207.2`, while the two-level header band (`Three Months Ended` / `Nine Months Ended` over `June 25, 2022` / `June 26, 2021`) sits at `y=169–186`. The extracted table begins at the row `Products`. Four columns of figures arrive **completely unlabelled**.

PyMuPDF's own `Table.header` does not rescue this. Measured on four sampled tables it was wrong on three:

Page	external	What it returned	Verdict
4	<code>False</code>	<code>['Products', '\$', '63,355', ...]</code>	First <i>data</i> row as the header
18	<code>True</code>	<code>['', 'June 25, 2022', ...]</code>	Right, but lost the group level
16 t0	<code>False</code>	<code>['Net sales', '\$', '37,472', ...]</code>	First data row again
16 t1	<code>True</code>	<code>[..., 'June 25, 2022 June 26, 2021', ...]</code>	Two dates merged into one cell

3. Columns are inflated two to three times over. `$` occupies its own column, `%` occupies its own column, and empty spacers pad between. Page 4: 12 columns for 5 logical ones. Page 18: 18 for 6. Roughly 60% of extracted cells are empty or hold a bare symbol.

4. Units live outside the table. `(In millions, except number of shares...)` sits in the narrative above it. A retrieved fragment stripped of that line invites reporting Apple's quarterly net sales as `\$82,959` rather than `\$82.96 billion` — a two-orders-of-magnitude error in a financial system.

The saving grace. Nine tables are introduced by a near-templated sentence — *"The following table shows net sales by category... (dollars in millions):"*. These carry exactly what extraction destroys: contents, periods, and units.

5. Architecture

```
PDF → Loader (boilerplate strip · printed-page resolver · section tagger)
  └─ Text   → section-aware chunking, table regions subtracted
  └─ Table  → header recovery ABOVE bbox → column collapse → units + caption
  │          → validation → dual representation (Markdown + structured rows)
  └─ Figure → image + bbox + caption + context + page render
    → Embeddings → Qdrant || BM25
    → RRF fusion + soft content-type routing
    → Context builder (dedupe · budget · provenance · injection fencing)
    → gpt-4o-mini | gpt-4o for visual questions and table fallback
    → Answer + citations + refusal flag + telemetry
```

Design principles, in priority order:

1. **Extraction correctness before retrieval sophistication.** No amount of reranking recovers a number whose column label was never indexed.
2. **Provenance is part of the data**, attached at extraction and carried unchanged.
3. **Honest capability reporting.** A thinly-evidenced pathway is described as one.
4. **No complexity the document does not justify.** The corpus is 70k characters.
5. **Degrade, don't crash.** Every stage has a defined fallback.

6. Data ingestion

Boilerplate detection is statistical, not hardcoded. Each candidate line at a page edge is normalised — whitespace collapsed, digit runs replaced with `#` — and lines recurring across many pages are classified as furniture. This solves two problems at once: `Apple Inc. | Q3 2022 Form 10-Q | 15` normalises to a constant so it is found automatically, and the digits it varies by *are* the printed page number.

A guard requires furniture to contain at least one letter. Without it, frequency analysis learns the forms `'#, #'`, `'$'` and `'@'`, which match bare currency values sitting at a page edge. Page 13 ends with the figure `53,325` directly above the footer, and stripping backwards from the page end deleted a real value from a financial statement. **This was caught by verification against the document, not by review** — it is the kind of defect that silently degrades a corpus.

Section structure is read from typography, because the PDF has no outline. Bold spans at 8.0 pt and above are headings; table labels are bold at 7.2 pt and are correctly excluded. Headings are indexed with their vertical position, so a page carrying several — page 14 has six, page 23 has three separate Items — resolves each piece of content to the heading actually above it.

7. Text processing

Uniform N-character splitting was rejected. Chunking follows section boundaries first, then paragraphs, packed to ~900 characters with ~150 overlap. Mean page text is ~2,500 characters, so this yields roughly three chunks per narrative page — fine enough for precise citation, coarse enough to preserve an argument.

Text blocks overlapping a table region by more than half are excluded. Without this, every figure in every table is indexed twice: once inside a normalised table with its period headers attached, and once as loose prose with none. The second copy is strictly worse and competes with the first at retrieval time.

A minimum chunk length is enforced. Section boundaries force a flush, which otherwise produces fragments of a few characters — a stray heading — that retrieve noisily and carry nothing a neighbouring chunk's breadcrumb does not.

8. Table processing

The heart of the project. Six ordered steps.

Header recovery

Fragments in the band above the table are collected, caption-width lines dropped, and the remainder grouped into columns by **x-overlap**, then merged top to bottom — a header is split across lines, **June 25,** above **2022**.

Group headers need different treatment. **Three Months Ended** is **centred over** its columns rather than spanning them; on page 18 its x-range overlaps only one of the three columns it heads. Assignment is therefore by **midpoint boundaries** between consecutive group headers, which places every leaf correctly.

The search is **floored at the bottom edge of any preceding table** on the page. A table's headers always lie between the previous table and this one, and without the floor page 16's second table absorbs the first table's final data row.

The row-label column is identified **geometrically**, by its left edge, so it stays unlabelled even on a table whose cell geometry is corrupt.

Column collapse

Raw columns sharing a recovered header form one logical column. This is what makes the **staggered layout** tractable — a discovery from testing, not from reading the PDF:

iPhone (1)	\$	40,665		\$	39,570	...
Mac (1)	7,382			8,235	...	

A value sits in a *different raw column* depending on whether a currency symbol precedes it. Raw column 1 holds **\$** for iPhone but **7,382** for Mac. Neither column is uniformly a symbol column, and neither alone holds a period's values — but per row, the group holds exactly one. Symbols are read per row, so a row the filing prints without **\$** does not gain one.

Parenthesised negatives become a leading minus: leaving (10) in place invites a sign error the moment a model does arithmetic on it.

Result

Page	Raw columns	Collapsed	Headers recovered
4 (Statements of Operations)	12	5	Both period groups, all four dates
18 (Net sales by category)	18	7	Both groups plus two Change columns
16 (Segment information)	12	5	Both period groups

Across all 31 tables: **336 raw columns collapse to 146 — 57% removed**. 27 validate clean, 11 captions and 25 units are attached.

Dual representation

`markdown` drives retrieval and display; `rows` gives an unambiguous `row_label → value` lookup, so "net sales for the three months ended June 25, 2022" resolves to one cell rather than a guess among four columns.

9. Figure processing

The document contains one image, and it is the Apple logo. Searching all 28 pages for `figure|chart|graph|diagram` returns seven matches, **every one of which is the word "Exhibit"**.

Three responses were available, and only one is defensible:

1. **Quietly redefine "figures" as tables.** Rejected — it misrepresents the system.
2. **Build nothing and state the document has none.** Rejected — the assignment asks for a capability, and absence of test data is not absence of a requirement.
3. **Build a genuine pathway and report the corpus honestly.** Adopted.

The implementation extracts images with bounding box, page render, surrounding narrative, and an `is_decorative` flag. The flag is the honesty mechanism: an uncaptioned image small enough to be a logo is marked, so the system never implies it interpreted a data graphic. A caption is claimed only when surrounding text actually names a figure — nothing in this filing does.

The page-render path is load-bearing twice over. It serves figure questions *and* acts as the fallback for tables that fail validation, where the rendered page shows the filing as published rather than a mis-parsed grid. The figure pathway is therefore not dead weight built to satisfy a checkbox.

A failure found in testing

The first figure query — "What images or figures appear in this document?" — retrieved only text chunks and the model then reported the exhibit certifications as images. Two causes: BM25 has no morphology, so `images` never matched `image`; and the figure chunk's embedding was diluted by cover-page prose.

Both were fixed, and a third change addressed the real issue: "what figures are in this document?" is a question about the **corpus**, which no per-figure chunk can answer. A document-level figure inventory chunk was added,

derived from measured extraction counts. The system now answers correctly:

The document contains 1 embedded image, which is decorative... There are no charts, graphs, diagrams, or data figures in the filing.

10. Retrieval strategy

Hybrid, because this corpus demands both. Dense retrieval handles paraphrase; BM25 handles the exact tokens that saturate financial queries — `Note 5`, `June 25, 2022`, `82,959` — where embeddings are weakest and a near miss returns the wrong number.

Fusion by Reciprocal Rank Fusion (k=60) rather than weighted score blending, because cosine and BM25 scores occupy different, corpus-dependent scales.

Routing is a soft bias, never a filter. A misrouted hard filter excludes the correct evidence unrecoverably; a misrouted bias only reorders.

A unit test exposed a flaw in the original implementation. RRF scores are compressed — at k=60, rank 1 scores 1/61 and rank 10 scores 1/70, barely 13% apart — so the initial boost of 0.15 could lift a rank-10 chunk above rank 1, contradicting a code comment claiming it could not. The bound is $r < 1 + (k+1) \cdot \text{boost}$; the default was corrected to 0.05, worth about three rank positions, and the real relationship documented.

11. Generation strategy

Two models, split by cost and risk:

Path	Model	Frequency	Reasoning
Text answers	<code>gpt-4o-mini</code>	Every query	Mostly lookup — the normaliser resolved labels and units upstream
Visual reading	<code>gpt-4o</code>	Figure questions, failed-table fallback	Rare, hard, and the last resort when structured extraction failed

Paying for capability only where it is scarce.

The system prompt spends most of its instruction budget on **refusal**, units, and period discipline, because declining is the behaviour most RAG systems get wrong and a smaller model is more prone to answering from world knowledge. Output is JSON with an explicit `refused` boolean, so a refusal is a distinct outcome rather than something a client infers from prose.

Prompt injection is mitigated architecturally: retrieved content is fenced, the system prompt states that text inside the fence is data to be quoted and never instructions, the generation call exposes no tools, and output is rendered as text. The practical risk on an SEC filing is negligible, but the ingestion path accepts arbitrary PDFs.

12. Citation and provenance

Every chunk carries `document_id`, both page numbers, Part, Item, Note, section, content type, and bounding box, attached at extraction and unchanged thereafter.

Citations lead with the **printed** page because that is what a reader sees and what the filing's own contents page uses, with the PDF index following for unambiguous lookup:

Sources:

– Page 15 (PDF p. 18) – table – Products and Services Performance

Getting this backwards would make every citation in the system off by three.

13. Evaluation methodology

33 questions across four categories, with ground truth read from the filing during extraction and cross-checked against rendered pages — **not generated by the system under test**.

Category	n	What it probes
Table	15	Exact lookup, comparison, multi-row reasoning, sign handling
Text	10	Factual, section-scoped, and multi-hop narrative
Figure	3	Inventory, absence of charts, visual description
Unsupported	5	Periods, companies, and topics the filing does not cover

No LLM judge is used. This is a decision, not an omission: most questions have a single numeric answer where a normalised substring check is exact and a judge would add its own error rate; the behaviour most worth measuring is refusal, which is already a boolean; and a judge sharing a provider with the system under test is not independent.

What that leaves unmeasured is stated plainly: correctness of 10 narrative answers with no single checkable value. They are scored on retrieval and refusal only. Fluency, completeness and groundedness of prose answers are not assessed automatically.

Retrieval is scored at **page granularity**, because a page yields several chunks and any may carry the answer. The cost is that precision is optimistic where a page contributes many chunks — reported rather than hidden.

14. Results

Two consecutive runs at identical settings, `top_k=8`.

Retrieval

Metric	Result	Interpretation
Hit rate@8	96.4%	An answer-bearing page reached the context in 27 of 28 scored questions
Recall@8	91.1%	Most questions retrieve all their answer pages
Precision@8	34.1%	Low by construction — see below
MRR	0.724	The first correct chunk is typically rank 1–2

Precision@8 deserves comment rather than concealment. With `top_k=8` and answers usually on one page, at most 1–3 of 8 retrieved chunks can be relevant, so ~34% is close to the practical ceiling. Recall and MRR are the meaningful measures here.

Answers

Metric	Result
Accuracy (23 checkable)	91.3%
Refusal accuracy	97.0%
Hallucination rate	0.0%
False refusal rate	3.6%
Citation accuracy	82.1%
Median latency	2.5 s

Category	n	Hit@8	Accuracy
Table	15	93.3%	86.7%
Text	10	100%	100% (3 graded)
Figure	3	100%	— (0 graded)
Unsupported	5	—	100%

All five unsupported questions were correctly declined, including "What were Apple's net sales in fiscal 2024?" — a fact the model certainly knows from pre-training.

Ingestion

28 pages → 135 chunks (102 text, 31 table, 2 figure) in ~25 s: extraction 13.9 s, embedding 5.0 s, indexing 3.8 s.
27 of 31 tables validate clean.

Run-to-run variance

Retrieval metrics were **identical** across runs. Answer accuracy moved between 91.3% and 95.7% across three runs at temperature 0 — a single question changing verdict. **The lower figure is quoted throughout this report;**

quoting the best run would misrepresent the system.

15. Design decisions

Full record in [docs/design-decisions.md](#) (D1–D11). The load-bearing ones:

Decision	Choice	Reasoning
D1	PyMuPDF; pdfplumber fallback dropped	Measured worse — see §16
D3	Qdrant over FAISS	Not justified by scale. Justified by payload filtering the architecture uses, and a Compose deployment that is substantive rather than decorative
D5	RRF over weighted blending	No per-corpus constant to tune and justify
D6	Two models	Opposite cost/quality profiles on frequent vs rare paths
D7	Reranking deferred	Plan asked whether it adds value; adopting it before measuring would have contradicted that
D8	Direct implementation over LangChain/LlamaIndex	The document-specific repair sits outside what any framework provides; a framework would wrap the hard part rather than solve it
D11	Merged-cell repair reverted	See §16

16. Challenges

The table headers — the central challenge

Covered in §4 and §8. Roughly half the project's effort.

Two remedies that measured worse than the problem

Page 16's second table extracts its nine-month columns as one cell, `$ 119,455 $ 106,048`. Two fixes were tried and both rejected **on evidence**.

pdfplumber, which the original design named as the fallback parser, is *worse* than PyMuPDF here:

Parser	Result
PyMuPDF	Two values merged into one cell — wrong shape, all data present
pdfplumber	Row label dropped, both nine-month values missing entirely

A merge is recoverable; a deletion is not. A fallback that loses data on the only case it exists to rescue is not a fallback, so D1 was revised.

Splitting the merged cell was then implemented, since the values are present rather than lost. It produced wrong numbers. That table's cell geometry is already corrupt — header assignment yields a duplicated period column and the row label absorbs a value, reading `'Segment operating income $ 31,583'` — so the split scattered figures into mis-ordered columns and **the table then passed validation while holding incorrect data**.

Caught by asserting the recovered *values* rather than the validation *status*. A confidently wrong table is worse than one marked untrustworthy, so the repair was reverted and the failure signature became a validation rule of its own (`COLUMNS_AMBIGUOUS`).

Tuning that made results worse

Change	Accuracy	Hallucination
Baseline (<code>top_k=8</code> , windows 20)	91.3%	0.0%
Candidate windows → 40	91.3%	0.0%
<code>top_k</code> → 12	87.0%	20.0%

Widening candidate windows changed nothing, because the binding constraint is the final `top_k` , not the candidate pool. Raising `top_k` to 12 was actively harmful: more marginally-relevant evidence gave the model more material to rationalise an answer from, and a negative question got answered. **More context is not better**, and on this system the cost was measured in hallucinations. Defaults were kept.

Silent data loss in boilerplate stripping

Described in §6. A real value deleted from a financial statement, found by verification against the document.

An ambiguous evaluation question

Testing showed the system answering `-13` where `-10` was expected. Investigation showed the *question* was at fault: the filing uses the label `Other income/(expense), net` twice — as a `-13` component inside Note 4, and as the `-10` income-statement total. The system's answer was defensible. The question was rewritten to name the statement, and the incident is recorded because it is exactly the "same term in multiple sections" hazard the PDF analysis had predicted.

17. Assumptions

- The assignment's test link refers to the AAPL 2022 Q3 10-Q from `docugami/KG-RAG-datasets` .
- "Figures" means images and data graphics; this filing has only the former, and the distinction is made explicit rather than elided.
- Printed page numbers are the right citation unit, with PDF indices alongside.
- The document fits comfortably in a modern context window. Retrieval is justified by precision, citation, and cost — **not by corpus scale**, and the design documents say so rather than implying a scale problem that does not exist.
- Ground truth in the evaluation set is correct; it was read from the filing and cross-checked against rendered pages.

18. Limitations

1. **One consistent retrieval failure.** "What was Apple's net income..." does not retrieve page 4: that table's embedding is diluted across 23 line items, ranking it 26th dense and 23rd lexical, outside the candidate window. The system **refuses rather than guessing** — correct behaviour on a retrieval miss — but the answer was retrievable.
 2. **The smaller model conflates same-labelled line items**, returning the `-13` Note 4 component rather than the `-10` statement total even when the question names the statement. A `gpt-4o-mini` limitation, reported rather than excused.
 3. **Citations are omitted on roughly one answer in five** (citation accuracy 82.1%). The interface falls back to showing all retrieved evidence.
 4. **The figure corpus is one decorative logo.** The pathway is complete and exercised, but thinly evidenced. No claim is made that data-figure interpretation is demonstrated at scale.
 5. **Four tables are flagged rather than resolved** — three with unresolved headers (two being the table of contents, which has none) and one with corrupt geometry. None are silently indexed.
 6. **Narrative answer correctness is not automatically measured.**
 7. **Reranking was never measured.** Deferred, not evaluated. Reported as unmeasured.
 8. **Extraction heuristics are tuned to this document's measured characteristics.** A different filing would warrant re-running the PDF analysis first.
-

19. Future improvements

Directly indicated by the measurements:

- **Cross-encoder reranking**, aimed at limitation 1. The diagnosis already exists: the correct chunk ranks 23rd–26th and needs promotion, which is exactly what a reranker does.
- **Row-level table representations** alongside the whole-table chunk, so a single line item is retrievable without splitting a table and severing its headers.
- **A stronger text model** for questions turning on same-labelled items, or a disambiguation step that surfaces both candidates and asks which is meant.

Broader:

- An **LLM judge using a different provider** for narrative answers, with limitations documented, to close the unmeasured gap.
 - **Multi-document support**, with per-document extraction profiles.
 - **Structured numeric answering** — parse the retrieved cell and compute, rather than asking the model to do arithmetic on text.
-

20. Conclusion

The system answers questions across all three required content types, grounds every answer in cited evidence, and declines when the filing does not support an answer — measured at a **0% hallucination rate** across five adversarial questions the model could easily have answered from pre-training.

The engineering that matters here is not the retrieval pipeline, which is conventional. It is the recovery of table structure that the PDF conversion destroyed: recovering period headers from outside the table's own bounds, collapsing a staggered symbol layout, and carrying units that live in the narrative. Without that work, a system over this document returns numbers with no period attached and is wrong roughly three times in four on period-specific questions — while looking entirely confident.

The parts that did not work are documented alongside the parts that did: a parser fallback that lost data, a cell repair that produced confidently wrong tables, and two tuning changes that degraded results. Those negative results were as expensive to obtain as the positive ones and are more useful to the next engineer.